Introduction

Palindrome problems show up constantly in coding interviews because they test something more specific than "can you loop through a string." They test whether you can spot structural symmetry and exploit it instead of brute-forcing your way through. LeetCode 3734 is a great example: on the surface it looks like a permutation-generation problem, which should terrify anyone who remembers how fast factorials grow. But the moment you notice that a palindrome is fully determined by its first half, the entire problem shrinks from "generate every arrangement" to "count letters and place them greedily."

This pattern — one half of a structure determining the other — appears far beyond palindromes. Symmetric matrices, mirrored UI layouts, and certain anagram-based problems all reward the same instinct: stop arranging, start counting. That's the core skill this problem is really testing, and it's why interviewers like it.

Problem Overview

You're given two strings, `s` and `target`, both of the same length, containing only lowercase letters. You need to rearrange the letters of `s` into a palindrome, then, among every possible palindrome you could build from those letters, find the lexicographically smallest one that is strictly greater than `target`.

If no rearrangement of `s` can form a palindrome that beats `target`, return an empty string.

Two things make this harder than a standard "smallest palindrome" problem:

1. Not every multiset of letters can even form a palindrome.
2. "Smallest palindrome greater than target" isn't the same as "smallest palindrome overall" — you sometimes have to deliberately choose a larger letter early on just to clear the target.

Example

Let's trace `s = "baba"` and `target = "abba"`.

Counting letters in `s`: two `a`'s, two `b`'s — both even, so a palindrome is possible (no letter needs to sit alone in the middle).

The palindrome's left half only needs one `a` and one `b` (the right half mirrors it). If we match `target`'s left half exactly, we'd get `ab` → mirrored → `abba`. But that **ties** `target`, it doesn't beat it. Since we need strictly greater, this doesn't count.

So we bump the left half up: the next viable left half using our letters is `ba`. Mirrored, that gives us `baab`.

Check: is `"baab" > "abba"`? Yes — `b > a` at the very first character. That's our answer.

This example already reveals the trap: matching the target exactly can produce a tie, and a tie is not a win.

Intuition

Here's how an experienced engineer would approach this without jumping straight to code.

Step 1: Notice the mirror. A palindrome of length `n` is completely determined by its first `[n/2]` characters. The back half is forced — it's just the reverse of the front (skipping the middle character if the length is odd). So instead of thinking "arrange the whole string," think "decide half the string."

Step 2: Figure out when a palindrome is even possible. For a string's letters to form *any* palindrome, at most one letter can have an odd count. That one odd letter, if it exists, must sit in the exact center. Every other letter needs to split evenly across the two mirrored halves. If more than one letter has an odd count, no palindrome exists — bail out immediately.

Step 3: Understand what "greater than target" really means here. Because the second half mirrors the first, the left half almost always decides the entire comparison — the first position where the built string differs from `target` determines the winner. The one exception: if the left half exactly matches `target`'s left half, then the outcome depends on the *whole* string, since `target` itself might not even be a palindrome.

Step 4: Build greedily, left to right. Try to match `target` exactly, character by character, using up letters from your available pool as you go. The moment you can't match a position (or you've matched everything but the result doesn't beat `target`), that's your signal to increase a letter somewhere.

Step 5: When you need to increase, increase as late as possible. At the current position, check if there's any available letter larger than `target`'s letter there. If yes, place the *smallest* letter that's still larger, then fill every remaining position with the smallest leftover letters possible. This guarantees the smallest valid result.

Step 6: If you can't increase here, back up. This is exactly like carrying a digit in addition. If position `i` has no larger letter available, move to position `i-1` and try there instead. If you back up past the very first position with no luck, no valid palindrome exists.

Brute Force Solution

Idea: Generate every unique permutation of `s`, filter for the ones that are palindromes, sort them, and pick the smallest one strictly greater than `target`.

Algorithm: 1. Generate all unique permutations of `s`. 2. Filter to only those that read the same forwards and backwards. 3. Sort the palindromes. 4. Return the first one greater than `target`, or empty string if none exists.

Advantages: - Trivial to reason about and verify correctness. - No edge-case handling needed — the definition does the work.

Disadvantages: - Catastrophically slow. Even with only 10 distinct letters, you're looking at over 3 million permutations before deduplication. - LeetCode caps `s` at length 300 — brute force isn't just slow here, it's completely infeasible.

Complexity: Time is O(n!) in the worst case (before filtering/deduplication), and space is O(n! · n) to store the permutations. Neither scales past very small inputs.

Optimal Solution

The optimal approach never generates a full string until the very end. It works entirely with letter counts and a single half-length array.

Step-by-step:

Step	What happens
1	Count every letter in `s` using a 26-length frequency table.
2	If more than one letter has an odd count, return `""` immediately — no palindrome is possible.
3	If exactly one letter is odd, remember it as the middle character. Halve every count to get the letters available for one half of the palindrome.
4	Take `target`'s left half (`target[:half_len]`) — this is what we're actually racing against.
5	**Exact-match attempt:** if the halved letter pool can spell `target`'s left half exactly, build it, mirror it, insert the middle letter, and check if the *full* palindrome beats the *full* target. If yes, that's the answer.
6	**Greedy build with backtrack:** walk left to right, matching `target`'s left half as far as possible, snapshotting the remaining letter counts at each position.
7	From the point where matching failed (or ended in a tie), walk backward. At each position, check if a strictly larger letter is available in that position's snapshot.
8	At the first (rightmost) position where a larger letter is available, place the smallest such letter, then fill the rest of that half with the smallest remaining letters in sorted order.
9	Mirror the completed half, insert the middle character if one exists, and return the result. If no backtrack position works, return `""`.

The key insight in step 5 is subtle but critical: matching `target`'s left half exactly doesn't guarantee your palindrome beats `target` — it might tie. Only checking the *fully mirrored* string against the *entire* target catches this.

Python Code

```
from collections import Counter


def smallest_palindrome(s: str, target: str) -> str:
    n = len(s)
    counts = Counter(s)

    odd_letters = [ch for ch, c in counts.items() if c % 2]
    if len(odd_letters) > 1:
        return ""

    middle = odd_letters[0] if odd_letters else ""
    half_len = n // 2
    half_counts = {ch: c // 2 for ch, c in counts.items()}

    target_half = target[:half_len]

    def fits(word: str, pool: dict) -> bool:
        need = Counter(word)
        return all(pool.get(ch, 0) >= cnt for ch, cnt in need.items())

    def smallest_from(pool: dict, length: int) -> str:
        letters = []
        for ch in sorted(pool):
            letters.append(ch * pool[ch])
        return "".join(letters)[:length]

    def build_result(left_half: str) -> str:
        return left_half + middle + left_half[::-1]

    # Attempt 1: exact match with target's left half.
    if fits(target_half, half_counts):
        pool = dict(half_counts)
        for ch in target_half:
            pool[ch] -= 1
        candidate = build_result(target_half)
        if candidate > target:
            return candidate

    # Attempt 2: greedy build with backtrack-and-carry.
    pool = dict(half_counts)
    snapshots = []
    matched_len = 0
```

```
    for i, ch in enumerate(target_half):
        snapshots.append(dict(pool))
        if pool.get(ch, 0) > 0:
            pool[ch] -= 1
            matched_len += 1
        else:
            break

    # Backtrack from the failure point (or the full length) to the start.
    for i in range(matched_len, -1, -1):
        snap = snapshots[i] if i < len(snapshots) else pool
        target_ch = target_half[i] if i < half_len else None
        bump_letter = None
        for ch in sorted(snap):
            if snap[ch] > 0 and (target_ch is None or ch > target_ch):
                bump_letter = ch
                break
        if bump_letter is None:
            continue

        left_half = target_half[:i] + bump_letter
        remaining_pool = dict(snap)
        remaining_pool[bump_letter] -= 1
        left_half += smallest_from(remaining_pool, half_len - i - 1)
        return build_result(left_half)

    return ""
```

Code Walkthrough

- `Counter(s)` builds the letter-frequency table in O(n).
- `odd_letters` collects every letter with an odd count; if there's more than one, no palindrome exists, so we return early.
- `middle` holds the lone odd-count letter (empty string if the length is even).
- `half_counts` divides every count by two — this is the letter pool available for one mirrored half.
- `target_half` is the portion of `target` we're actually comparing against, since the second half is forced.
- `fits` checks whether a candidate word can be spelled using a given letter pool — used to test the exact-match shortcut.
- `smallest_from` lays out whatever letters remain in sorted order, truncated to a needed length — this is always the lexicographically smallest arrangement of those letters.
- `build_result` mirrors a left half around the middle character to form the full palindrome.
- The exact-match block tries `target_half` directly; if it's spellable and the resulting full palindrome beats `target`, we're done.

- The greedy block matches `target_half` position by position, saving a snapshot of the pool *before* consuming each letter, so we know exactly what was available if we need to backtrack to that position.
- The final backward loop is the "carry" step: starting from the last matched (or failed) position and walking back to the very start, it looks for the first position where a strictly larger letter than `target`'s is available, places it, fills the rest minimally, and returns.

Dry Run

Using `s = "baba"`, `target = "abba"`:

1. `counts = {'b': 2, 'a': 2}`, no odd letters, `middle = ""`, `half_len = 2`.
2. `half_counts = {'b': 1, 'a': 1}`, `target_half = "ab"`.
3. **Exact match:** `"ab"` fits `{'b':1,'a':1}`. Build: `"ab" + "" + "ba"` = `"abba"`. Compare: `"abba" > "abba"` is `False` (a tie). Exact match fails.
4. **Greedy build:** position 0, target char `'a'`, pool has `a`. Snapshot `{'b':1,'a':1}` saved, consume `'a'` → pool `{'b':1,'a':0}`. Position 1, target char `'b'`, pool has `b`. Snapshot `{'b':1,'a':0}` saved, consume `'b'` → pool `{'b':0,'a':0}`. `matched_len = 2`.
5. **Backtrack from i=2** (full match, no failure): no snapshot at index 2, use final `pool = {'b':0,'a':0}`, `target_ch = None`. No letters available — skip.
6. **i=1:** snapshot `{'b':1,'a':0}`, `target_ch = 'b'`. Sorted letters: `'b'` — is `'b' > 'b'`? No. No bump available — skip.
7. **i=0:** snapshot `{'b':1,'a':1}`, `target_ch = 'a'`. Sorted letters: `'a'`, `'b'`. `'a' > 'a'`? No. `'b' > 'a'`? Yes — bump letter is `'b'`. `left_half = "" + "b" = "b"`. Remaining pool after removing `'b'`: `{'b':0,'a':1}`. Fill remaining `2 - 0 - 1 = 1` slot with smallest: `"a"`. `left_half = "ba"`.
8. `build_result("ba")` = `"ba" + "" + "ab"` = `"baab"`.

Result: `"baab"`, matching our hand-traced answer.

Complexity Analysis

Time: O(n) for counting letters, plus O(n) for the greedy matching pass, plus O(n · 26) in the worst case for the backtrack loop (at each of up to n/2 positions, scanning up to 26 letters), plus O(n log n) for sorting in `smallest_from` — but since there are only 26 possible letters, sorting is effectively O(26 log 26), a constant. Overall this is **O(n)** with a constant factor of 26.

Space: O(n) for the snapshots list (one dictionary of size ≤26 per position) plus O(n) for the output string. Effectively **O(n)**.

Why this is correct: The palindrome constraint means only the first half needs to be decided, cutting the search space from "all permutations" to "all arrangements of half the letters." The greedy-with-backtrack strategy mirrors how you'd increment a number to find the next value larger than a target: match digits exactly as long as possible, and the first position (from the right) where a larger digit is available determines the answer. Every position after the bump can be filled with the smallest available letters because any valid combination beats `target` from that point on — there's no need to optimize further right.

Alternative Solutions

One alternative is sorting the entire letter pool and mirroring it directly, without any target-awareness. This gives you the *smallest possible palindrome* from `s`'s letters — but it ignores the "greater than target" constraint entirely. It only works by coincidence if that smallest palindrome happens to already beat `target`. The greedy-with-backtrack approach subsumes this: it naturally falls back to the smallest arrangement whenever no target-matching constraint is active (see `smallest_from`).

Another alternative is binary-search-like reasoning over candidate left halves, but since letters aren't numeric and counts are constrained, this doesn't simplify beyond the greedy approach — it just re-implements it with more overhead.

Edge Cases

- **All letters identical** (e.g., `s = "aaaa"`): only one arrangement is possible; either it beats `target` or the answer is `""`.
- **Odd-length string with all-even letter counts:** impossible to form any palindrome — the middle character has no candidate. Must return `""`.
- **`target` already the maximum possible palindrome** from `s`'s letters: no larger one exists, return `""`.
- **`s` and `target` are anagrams of each other and `s` happens to already be `target`:** exact match will tie, forcing the greedy path.
- **Single-character strings:** trivially a palindrome; compare directly against `target`.
- **More than one letter with an odd count:** immediately invalid, return `""` before doing any other work.

Common Mistakes

1. **Forgetting the "at most one odd count" check** and trying to build a palindrome from letters that can't form one.
2. **Assuming an exact match with `target`'s left half is automatically a win** — it can produce a tie, which fails the "strictly greater" requirement.

3. **Sorting all letters and mirroring without checking against `target`** — this gives the smallest palindrome overall, not the smallest one greater than `target`.
4. **Not saving snapshots during the greedy match**, making backtracking impossible without re-deriving the pool state at each position.
5. **Bumping too early instead of as late as possible** — always try positions from the right first, since bumping later in the string yields a smaller overall result.
6. **Mishandling the middle character** when the length is odd — forgetting to insert it, or using it in half-length calculations incorrectly.
7. **Off-by-one errors in half-length** when the total length is odd vs. even.

Interview Questions

- What changes if you need the *largest* palindrome smaller than `target` instead?
- How would you extend this to a case-insensitive or Unicode alphabet?
- Can you solve this without the exact-match special case, folding it into the backtrack loop?
- How would you handle multiple queries against the same `s` with different `target` values efficiently?
- What's the smallest number of letter swaps needed to go from `s`'s current arrangement to your answer?

Similar Problems

- **LeetCode 267 – Palindrome Permutation II:** generates all distinct palindromic permutations of a string; foundational for understanding the half-string trick.
- **LeetCode 266 – Palindrome Permutation:** checks whether *any* permutation of a string can form a palindrome, using the same odd-count-parity rule.
- **LeetCode 60 – Permutation Sequence:** a different "find the Nth/next" permutation problem that also benefits from avoiding full enumeration.
- **LeetCode 31 – Next Permutation:** the general-purpose version of "find the next lexicographically greater arrangement," minus the palindrome constraint — the backtrack-and-carry logic here is a close cousin.

Key Takeaways

A palindrome is defined by its first half — never arrange the whole string when you only need to decide half of it. Counting letters replaces generating permutations, turning a factorial problem into a linear one. When a "greater than target" constraint is involved, match exactly as long as possible, then increase the *latest* possible position and fill the rest minimally — exactly like carrying a digit in arithmetic. And always double-check: an exact match is not automatically a win, since it can tie instead of beat the target.

Watch the Video

If you want to see this traced out visually, position by position, with the backtrack step shown live, watch the full walkthrough here: {LINK}. It also includes real timing comparisons against brute force and two quiz rounds to test your understanding.

About the Series

This breakdown is part of **Fun with Learning Technology**, a series dedicated to building real algorithmic intuition instead of memorized solutions. Every episode picks apart a coding interview problem, works through the reasoning an experienced engineer would actually use, and backs it up with real code and real timing data — no hand-waving, no shortcuts skipped.

Call To Action

If this helped the problem click, drop a like on the video and subscribe for more daily breakdowns like this one. For deeper written walkthroughs delivered straight to your inbox, subscribe to the Substack. And if you spotted a cleaner approach or want the next follow-up question tackled, leave a comment — shares help this reach more developers prepping for interviews.

Quick Review

Pattern: fix a global property (letter counts) then greedily choose the smallest valid arrangement — trigger?

When arrangement is constrained by a fixed multiset (palindrome permutation), count letters first, then greedily build position-by-position instead of enumerating permutations.

Time complexity for building the palindrome greedily over the alphabet?

O(n × 26) roughly — n halves of the string, each position tries up to 26 letters against remaining counts.

Space complexity for this approach?

O(n + 26): output string/array of length n plus a fixed-size letter-count table.

Trickiest bug: comparing candidate to target for 'greater than'?

Must handle the tie-breaking prefix match carefully — a prefix equal to target isn't 'greater'; only diverge once a placed letter exceeds target's at that position.

Palindrome constraint vs plain permutation-greater-than problems — key difference?

Only the first half is freely chosen; the second half is mirrored, so parity of letter counts (at most one odd count) must be validated upfront.

Interview follow-up: how to extend for k-th smallest palindrome > target instead of just the next one?

Combinatorially count valid completions per prefix choice (like a digit-DP), skipping k of them per position instead of taking the first fit.
